LESSON 04 · MISSION GUIDE

The Roster

Columns, rows and lookups: lists and dictionaries, Python's containers for real data.

From **Quartermistress Pell**
To **Jen, Keeper of the Guild Ledger**
By raven **Where do the heroes go?**

Keeper, your loot function is lovely and I've used it eleven times. But to run it for the whole guild I'm typing names and levels into a text with slashes in it, and Orrin looked over my shoulder and made the face.

In Excel I'd have a column of rewards, a row per hero, and a lookup table on the side. Orrin says Python has all three, called a list, a dictionary, and "also a dictionary". Then he said "you'll see" and went to rescue the notice board from Biscuit. I would like to see. Captain Siv wants heroes per class by the weekend, so this is urgent-ish. — P.

What you'll be able to do after today

- [ ] Keep many values in a **list**: index it, slice it, add and remove, sort it, total it
- [ ] Keep one record in a **dictionary**: look up by name instead of by position, add and change fields
- [ ] Hold a whole table as a **list of dictionaries** and loop over it to total, filter and find the max
- [ ] Count things with a dictionary (COUNTIF without the ranges) and look things up (XLOOKUP without the #N/A)

Why this matters at work

- **This is what a spreadsheet is underneath.** A column is a list. A row is a dictionary. A sheet is a list of rows. pandas (lesson 09) is built on exactly this picture, so today's shapes are tomorrow's DataFrame.
- **Lookups that don't break.** An XLOOKUP that can't find a key gives `#N/A` and a wrong SUM further down. A dictionary's `.get` gives you the fallback you asked for.

Four big ideas

1. List = ordered, by position	`heroes[0]`, `heroes[-1]`, `heroes[1:3]`: the same positions and slices as letters in a string. `len`, `sum`, `max`, `sorted` work on them.
2. Dictionary = labelled, by key	`hero["level"]`. Keys are usually text; values are anything. Missing key → KeyError, unless you use `.get(key, fallback)`.
3. They nest	A list of dictionaries is a table. A dictionary of lists is a grouped table. `for hero in party:` then `hero["name"]`.
4. Build with append	Start with `[]` or `{}` before the loop, add inside, use after. The loop recipe from lesson 02, with a container instead of a total.

Three habits that will bite you

- `heroes.sort()` and `heroes.append(x)` change the list and return `None`. `print(heroes.sort())` prints `None`. Use `sorted(heroes)` for a copy.
- Dictionary keys are case-sensitive and exact: `"Ironhills"` and `"ironhills"` are different keys. Ask Dalvric.
- Two kinds of quotes in an f-string: `f"{hero['name']}"`. Same kind twice ends the string early.

How today works

15 MIN
Lecture
Lars builds a column, a row and a table live in the REPL

60 MIN
Exercise sheet
Seven ravens. You write every file yourself

15 MIN
Grimoire, page 4
The roster with one hero per class

The ravens

#	FROM	RAVEN	YOU'LL USE
1	Pell	The roster	list, index, `append`, `in`, `remove`, slice
2	Queen Aldara	A column of rewards	`sum max min len sorted`, filter into a new list
3	Orrin	One hero, many fields	dict, `[key]`, `.get`, add a field, loop keys
4	Pell	A list of dictionaries	a table: loop rows, total, max, filter
5	Captain Siv	Counting with a dictionary	the COUNTIF loop
6	Pell	XLOOKUP, the Python way	`.keys() .values() .items()`
7 ⭐	Queen Aldara	Crowns per day, by monster	dictionary of lists, functions

You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if you're feeling brave), each printing the expected output
- [ ] Dalvric's roster finds Wren, finds the Ironhills' warden, and counts the Warriors twice, with one commit per bug
- [ ] Everything is pushed to `lesson-04` and you've opened a pull request



Next lesson: The Scriptorium Scroll. You've been typing the roster into your scripts by hand. The Scriptorium has copied the notice board out as a CSV (badly); next time you'll open files, read them line by line, split them into lists and dictionaries, and write clean results back out.